

SOC Platform AI SIEM Complete Guide

Real-world attack monitoring, log analysis, alerts, incidents, AI Analysis, rules, and database access.

Generated for project: C:\Users\bpava\OneDrive\Documents\New project\Wazuh-SIEM-Security-Lab\siem-platform

1. Platform Overview

Your SOC Platform AI SIEM is a full-stack security monitoring application. The frontend shows Dashboard, Logs, Alerts, Incidents, AI Analysis, Rules, and Settings. The backend receives logs, stores records in PostgreSQL, evaluates rules, creates alerts, supports incidents, and saves AI analysis history.

- Frontend: React + Vite SOC dashboard.
- Backend: Node.js + Express API.
- Database: PostgreSQL for users, logs, rules, alerts, incidents, AI analyses, and audit records.
- Redis: queueing, rate limits, temporary counters, and real-time statistics.
- Detection: pattern, regex, field match, threshold, and correlation rules.

2. Start the Platform

```
cd "C:\Users\bpava\OneDrive\Documents\New project\Wazuh-SIEM-Security-Lab\siem-platform"
docker compose up -d postgres redis
docker compose run --rm backend npm run migrate
docker compose run --rm backend npm run seed
docker compose up -d
```

Open the application in your browser:

- Frontend: <http://localhost:5173>
- Backend health: <http://localhost:3001/health>
- Default admin email: admin@siem.local
- Default admin password: changeme123

3. Prove Real-World Attack Monitoring

Use the simulator to prove that ingestion, detection, alerting, and dashboard updates work end to end.

```
.\tools\simulate-attacks.ps1
```

The simulator logs in to the API, creates missing demo rules, sends attack logs, waits for processing, and prints a run id for evidence tracking.

- Brute force login attempts.
- Successful login after repeated failures.
- SQL injection against a web asset.
- Credential dumping indicator from endpoint logs.

Expected UI proof: Dashboard counters change, Logs page shows the run id, Alerts page shows triggered detections, Incidents can link the evidence, and AI Analysis can summarize the case.

4. Manual Log Ingestion

Authenticate first and store the JWT token:

```
$login = Invoke-RestMethod -Method Post -Uri "http://localhost:3001/api/auth/login"
-ContentType "application/json" -Body (@{
```

```
email = "admin@siem.local"
password = "changeme123"
} | ConvertTo-Json)

$token = $login.access_token
$headers = @{ Authorization = "Bearer $token" }
```

Send one SQL injection log:

```
Invoke-RestMethod -Method Post -Uri "http://localhost:3001/api/logs/ingest/single" -Headers
$headers -ContentType "application/json" -Body (@{
source_api_key = "firewall_demo_key"
log = @{
severity = "CRITICAL"
category = "web"
message = "WAF blocked SQL injection: /login?id=1 UNION SELECT password FROM users"
ip_src = "198.51.100.77"
ip_dst = "10.0.0.80"
user_name = "anonymous"
host_name = "public-web-01"
}
} | ConvertTo-Json -Depth 10)
```

Send brute force logs in bulk:

```
$logs = @()
for ($i = 1; $i -le 10; $i++) {
$logs += @{
severity = "HIGH"
category = "authentication"
message = "failed login for user admin from 203.0.113.50 attempt=$i"
ip_src = "203.0.113.50"
ip_dst = "10.0.0.25"
user_name = "admin"
host_name = "vpn-gateway-01"
}
}

Invoke-RestMethod -Method Post -Uri "http://localhost:3001/api/logs/ingest" -Headers
$headers -ContentType "application/json" -Body (@{
source_api_key = "firewall_demo_key"
logs = $logs
} | ConvertTo-Json -Depth 10)
```

5. How to Analyze Logs

- Open Logs and filter by severity: CRITICAL and HIGH first.
- Filter by source: firewall-main for network/web logs, endpoint-edr for host logs.
- Search by IOC: attacker IP, username, host name, URL path, payload, or simulator run id.
- Review timestamp, source, severity, category, message, ip_src, ip_dst, user_name, host_name, and extra_data.
- Decide if the event is true positive, false positive, or needs more evidence.

6. Alerts Workflow

The alert engine evaluates each ingested log against enabled rules. When a rule matches, the platform creates or updates an alert and streams it to the frontend.

- pattern: text match against a selected field.
- regex: regular expression match against the message.
- field_match: contains, equals, or regex against any log field.
- threshold: repeated matching events in a time window.
- correlation: suspicious login success after repeated failures from same source and user.

Alert triage steps: open Alerts, review severity/detail/source/log IDs, acknowledge the alert, create an incident if confirmed, then resolve only after response is complete.

7. Incident Workflow

- Create an incident for confirmed compromise, repeated activity, or multiple related alerts.
- Use a clear title such as SQL Injection Attempt Against public-web-01.
- Set severity based on the highest confirmed alert.
- Link alert IDs and log IDs as evidence.
- Add timeline notes for detection, validation, containment, eradication, recovery, and closure.
- Close the incident only after evidence and response actions are documented.

8. AI Analysis

AI Analysis supports SOC-style investigation. If ANTHROPIC_API_KEY is empty, the app records offline fallback analysis. Add the key in .env and restart backend for real AI output.

- triage: prioritize alert handling.
- forensics: reconstruct likely attack chain.
- ioc: extract IPs, users, hosts, URLs, and suspicious strings.
- threat_hunt: recommend additional searches.
- incident: create an incident response summary.

Example AI input:

Alert: SQL Injection Pattern triggered

Severity: CRITICAL

Source: firewall-main

Log: WAF blocked SQL injection: /login?id=1 UNION SELECT password FROM users

ip_src: 198.51.100.77

ip_dst: 10.0.0.80

host: public-web-01

Question: Triage this alert, map MITRE ATT&CK, list IOCs, and recommend response steps.

9. Rules

Rules define how the SIEM detects attacks. Start with a simple rule, test it with sample logs, then tune threshold, field, or regex to reduce false positives.

Seeded rules include Brute Force Login Attempts and SQL Injection Pattern.

Pattern rule example:

```
{
  "name": "Malware Keyword Detected",
  "severity": "HIGH",
  "type": "pattern",
  "conditions": {
    "field": "message",
    "operator": "contains",
    "value": "malware detected"
  },
  "actions": [{ "type": "create_alert" }],
  "is_enabled": true
}
```

Threshold rule example:

```
{
  "name": "Password Spray",
  "severity": "HIGH",
  "type": "threshold",
  "conditions": {
    "field": "message",
    "operator": "contains",
    "value": "failed login",
    "threshold": 10,
    "timeWindowSeconds": 60
  },
  "actions": [{ "type": "create_alert" }],
  "is_enabled": true
}
```

10. Database Access

The app stores records in PostgreSQL running inside Docker.

- Database: siem_db
- User: siem_user
- Password: POSTGRES_PASSWORD from .env
- Host from laptop: localhost
- Port: 5432

Enter the database terminal:

```
docker compose exec postgres psql -U siem_user -d siem_db
```

Show tables inside PostgreSQL:

```
\dt
```

Show records:

```
SELECT * FROM users;
SELECT * FROM log_sources;
SELECT * FROM logs ORDER BY ts DESC LIMIT 20;
SELECT * FROM alerts ORDER BY created_at DESC LIMIT 20;
SELECT * FROM incidents ORDER BY created_at DESC LIMIT 20;
SELECT * FROM rules ORDER BY created_at DESC;
SELECT * FROM ai_analyses ORDER BY created_at DESC LIMIT 10;
```

Exit PostgreSQL:

```
\q
```

11. Useful One-Line Database Commands

```
docker compose exec -T postgres psql -U siem_user -d siem_db -c "SELECT id, ts, severity, source_name, category, message, ip_src, user_name, host_name FROM logs ORDER BY ts DESC LIMIT 20;"
```

```
docker compose exec -T postgres psql -U siem_user -d siem_db -c "SELECT id, title, severity, status, source_name, occurrence, created_at FROM alerts ORDER BY created_at DESC LIMIT 20;"
```

```
docker compose exec -T postgres psql -U siem_user -d siem_db -c "SELECT id, name, severity, type, is_enabled, mitre_tactic, mitre_tech FROM rules ORDER BY created_at DESC;"
```

12. How the Database Works

Frontend UI sends requests to the Backend API. The backend validates input, applies authentication and role checks, then writes or reads records from PostgreSQL. Redis assists with queueing, rate limits, and temporary dashboard counters.

- users: login accounts and roles.
- log_sources: firewall/endpoint source identities and hashed API keys.
- logs: raw and normalized SIEM events.
- rules: detection logic.
- alerts: detections generated from matching rules.
- incidents: SOC cases created from alerts/logs.
- ai_analyses: saved AI outputs and offline fallback records.
- audit_log: important admin and analyst actions.

13. Evidence Checklist for Final Demo

- Dashboard before and after simulation.
- Simulator terminal output with accepted logs and run id.
- Logs page filtered by the run id.
- Alerts page showing triggered detections.
- Alert acknowledgement screenshot.
- Incident with linked alert/log evidence and timeline.
- AI Analysis output for selected alert or incident evidence.
- Rules page showing enabled detection logic.
- Database terminal showing rows in logs, alerts, incidents, rules, and ai_analyses.